

CLAUDE OS LECTURE

나만의 Claude OS 만들기

스킬·에이전트·혹으로 나만의 AI 운영체제 설계하기

DAY 1 / 4

Day 1

스킬 · 에이전트 · 오케스트레이터

DAY 1

강의 소개

이 강의의 목표



지식

지금까지의 **AI 엔지니어링 개념** 빠르게 따라잡기



경험

나만의 클로드 OS를 직접 만들며 핵심 개념 익히기



관점

AI에 대한 **다양한 입장과 실제 사례**를 공유하며
시야 넓히기

실습 위주 + 다양성 확보



시간 배정 (이론 30% + 실습/리뷰 70%)

15분 개념 + 25분 실습 + 10분 리뷰 + 10분 휴식



조별 활동 (5명 1조)

서로 리뷰하고 도움주기 · 다양성 추구

권장



완성보다 체험을 목표

고민하며 경험했으면 충분히 성공, 그 이상은 과제를 하며 연습 권장



적극적 소통

같은 목표, 다른 접근·시각이야말로 큰 가치가 있음

DAY 1

에이전틱 기초

스킬 / 에이전트 개념

재활용 가능한 단위

물리적으로는 .md 파일 한 개

기존 클래스/펄션과 유사

직관적으로 초기에는 md 파일을 클래스와 유사하게 생각해도 무방

직접 작성할 필요 없음

클로드 코드에게 "스킬/에이전트 만들어줘"라고 얘기하는 것으로 충분

SKILL.MD 예시

```
---
description: 로컬 변경사항을 Git에 커밋
---

# /commit

# 1) 변경사항 확인
git status
git diff

# 2) 커밋 메시지 작성 후 실행
git commit -m "feat: ..."
```

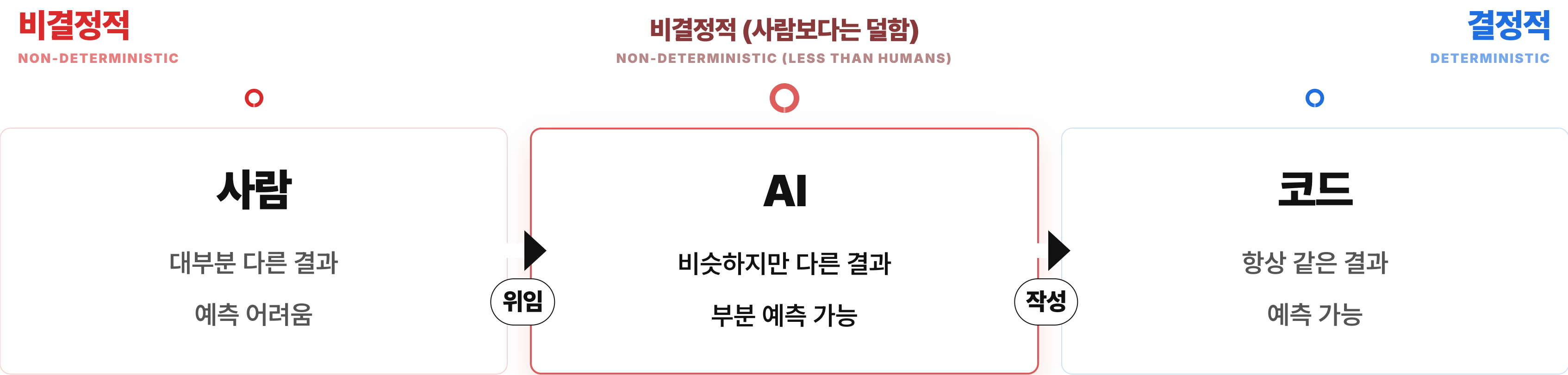

스킬 vs 에이전트

SKILL		SPAWN →	AGENT	SPAWN →	AGENT
스킬		에이전트			
호출	/xxx	스킬/에이전트가 spawn			
실행	포어그라운드	백 / 포어그라운드 모두			
병렬 실행	N/A	✓ 가능			
독립 컨텍스트	N/A	✓ 보유			

DAY 1

AI 리더십

AI 시대의 협업 구조



그동안 **엔지니어링** 으로 코드를, **리더십** 으로 사람을 관리했듯,
이제 **AI 리더십** 으로 AI를 관리해야 함

AI 리더십



대화하기

사람에게 말하듯 **질문 · 상의 · 요구하기**

AI 회의 시스템 사례 — "AI에게 부탁해 만든 것"



이해하기

AI가 **무엇을 잘하고 무엇을 못하는지** 익히기

대화 경험이 쌓이며 자연스럽게 강약점 파악



협업하기

잘하는 부분은 위임, 못하는 부분은 도와주기

2일차부터 다루는 주제

제가 경험한 AI (단점 위주)

지침

≠

AI 행동

지침 무시

명확한 지침이 있음에도 본인 판단을 더 우선시 함



중도 포기

문제를 풀다 길어지면 중간에 포기하려고 함

입장 A

↻

입장 B

입장 반복

다른 의견 한 마디에 너무 쉽게 입장을 180도 바꿈

스킬과 에이전트

사람과 대화하듯 만들어달라고 요구하면 됨



💡 리마인드: 사람과 대화하듯 만들어달라고 요구하면 됨 . 예) "git에 커밋하는 스킬 만들어줘"

01 첫 번째 스킬 · commit 만들기

로컬 변경 사항을 Git 커밋해주는 스킬 → 방금 만든 스킬로 커밋해보기

02 두 번째 스킬 · tech-qna

질문하면 답변해주는 스킬 → 클로드 코드 / 개발 관련 간단한 질문 답변 받아보기

03 첫 번째 에이전트 · 기존 tech-qna 스킬 수정

에이전트로 답변 후 팩트 체크 제공 → 질문 받아보기 → 실행 로그로 에이전트 로딩 관찰

DAY 1

훅 (Hook)

훅 (Hook)

스킬·에이전트·도구 라이프사이클 중 **특정 이벤트 발생 시 자동 트리거**

LIFECYCLE EVENTS 예시 (일부만 소개)



활용 영역 이후 다룰 핵심 도구

- 컨텍스트 엔지니어링 (2일차 주제)
- 하네스 엔지니어링 (4일차 주제)

흑으로 스킬 통계 기록 시스템 구축하기

흑을 활용해 사용한 스킬의 호출 횟수 기록 → skill-stat 스킬로 통계



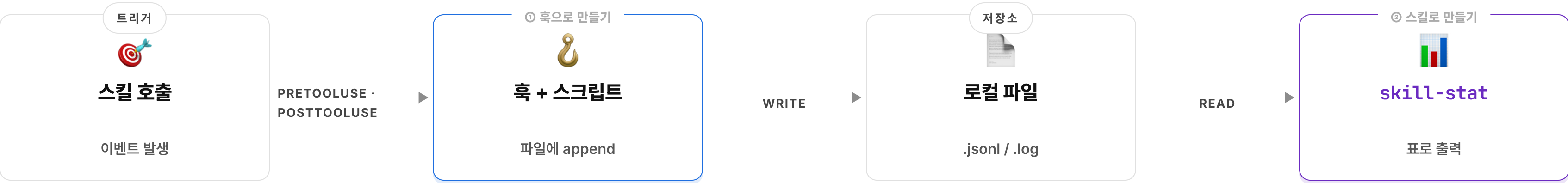
01 흑으로 스킬 호출 횟수·소요 시간 기록

사용자가 사용한 스킬의 호출 정보를 흑으로 기록

⚠ 이 부분은 스킬·에이전트 없이 순수하게 흑 + 스크립트 로 구현

02 skill-stat 스킬 만들기

위에서 쌓은 스킬별 통계를 CLI에서 표로 간단히 볼 수 있게 만들기



DAY 1

OS

OS

개인 혹은 팀의 목표를 원활히 달성하려고 만든 체계

구 성	
물리적	스킬 · 에이전트 · 혹은 등의 집합
논리적	관점 · 작업 절차 · 규칙 · 상호작용 · 권한 등의 집합

💡 강사의 OS: 스킬 58개 + 에이전트 33개 규모로 운영 중 (2026-05-21 기준)

제 OS 일부 소개

Business

외부 입력

/news

AI 비즈니스/기술 뉴스·경쟁사 동향

/research

팩트 기반 시장 데이터 조사

/advisor

스타트업 전략 결정·조언

Development

개발 워크플로우 (선형)

/spec

요구사항 인터뷰·문서화

→

/task

태스크 수행 (worktree 병렬)

→

/test

모든 테스트, 자동 수정

/deploy

배포

OS

메타 · 실험

/os

모든 스킬/에이전트 수정 (메타)

/ab-test

A/B 테스트 자동 수행

시스템 루프

지속적으로 우리 비즈니스를 개선하는 장치

Retro

OS 자율 개선

SSOT(Single Source of Truth)

도메인 문서 관리

Cover & Modify (Red Test First)

양질의 테스트 확보

💡 이 프레젠테이션도 /deck 스킬로 제작

SHOWCASE

20 / → / Space

OS를 구축하며 했던 시행착오



👉 시행착오를 하며 제가 선택한 접근법은 2일차 이후 주제를 통해 공유드릴 예정

나만의 OS 설계

OS 설계 후 README.md 기록해보기



💡 규모와 관계없이 내가 효능감을 느낄 수 있는 OS 추천. 예) 작업 시간 줄이기 · 귀찮음 줄이기 · 더 잘하기

01 내 업무의 중심 워크플로우는 무엇인가?

본인이 가장 많은 시간을 보내는 워크플로우를 OS에 녹여보기

02 나만의 OS에 꼭 포함되어야 하는 핵심 철학/관점은 무엇인가?

예) 주도 개발(TDD, DDD, SDD, BDD 등), 리눅스 파이프라인 철학(IO)

03 내 OS가 일을 잘 하려면 어떤 정보(컨텍스트)가 필요한가?

예) 개발 아키텍처, 테스트 정책, 코드 컨벤션 등

👉 위 3가지 생각을 README.md에 기록해주세요!

조별 인사

서로 편하게 나누기



SHARE 01

자기 소개 + 현재 업무 등 편하게 나누기



SHARE 02

본인이 만들고 싶은 OS 소개하기



DAY 1

오케스트레이터

오케스트레이터

01 개념

스킬과 여러 에이전트가 협력하여 목표를 달성 (보통 워크플로우 구현)

OS를 만들다 보면 자연스럽게 등장할 것

02 자주 보는 패턴

이터레이션 / 스프린트	기획 → 개발 → QA (Playwright)
TDD	Test First (Red) → 구현 → Test (Green) → 리팩토링 → Test (Green)

나만의 오케스트레이터 만들기

본인 OS에 어울리는 워크플로우를 정의하고 직접 구현



01 나만의 OS에 어울리는 워크플로우 만들어보기

어떤 단계를 어떤 순서로 묶을지 본인 OS 맥락에서 결정

02 만든 오케스트레이터 사용해보며 아쉬운 부분 수정

실제로 한 번 돌려보고 부족한 단계 보강

생각이 잘 안 난다면 · **/sprint** 오케스트레이터 구현하고 테스트 하며 개선해보기

인터뷰 (Agent) → 구현 (Agent) → 코드리뷰 (Agent) → 피드백 반영 (Agent) → 완료 보고

오케스트레이터 리뷰

함께 나누기



SHARE 01

어떤 이유로 어떤 오케스트레이터를 만들었는지



SHARE 02

앞으로 어떻게 만들어나가고 싶은지



1일차 정리

- 01 대화형 AI는 메시지마다 토큰이 누적되는 구조
- 02 정해진 용량을 컨텍스트 용량이라 부름
Opus 4.7: 0.2M / 1M
- 03 누적될수록 **Context Rot** 증상이 생기며 성능 저하
- 04 작업 후 즉시 `/clear`로 새 세션 시작 권장



👉 2일차에는 이 주제를 다루는 컨텍스트 엔지니어링 이야기

DAY 1 · 과제

내 OS 스킬 10개 이상 만들어 압축파일 제출

에이전트 + 오케스트레이터 활용 · 제출 방식은 슬랙으로 안내